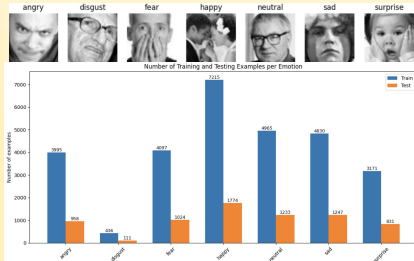


Facial Emotion Classification Model Based on KNN and CNN

Xun Lu, Austin He, Keyu Zhu

Introduction

Facial recognition has been an important topic in computer vision. In the final project, the database we used is Fer2013. Lots of models in Github have accuracies around 0.68 and accuracies over 0.7 are the top 10 in Trending Papers ranking. This database contains 28273 images(48X48) for training, 7067 images for testing showing 7 different kinds of emotion. Some examples and the distribution of it are as follows:

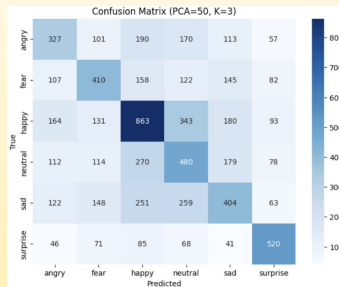
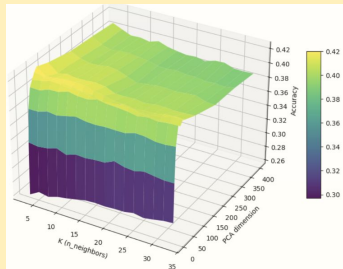


The goal of this project is to use KNN and CNN to see how high the accuracy could be in image classification.

KNN

Before training, we normalize the data to prepare for PCA. For training methodology, we considered the number of components in PCA as a hyperparameter for KNN. Then we did a grid search to loop through different pairs of them to find the best n and best K .

When $k = 3$ and dimension = 50, the accuracy is the highest, which is 0.4258.



From the confusion matrix above, KNN has the best performance in happy and surprise while it is bad at angry, fear, etc. One reason could be that the happy and surprise share some similarities and the rest also share some similarities. Another one could be that happy has most images so it influences the voting.

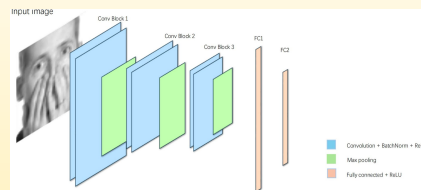
Although the accuracy of "happy" and "surprise" is relatively high, the overall accuracy is not ideal. Because the KNN model is simple, it cannot capture the complex features of images. And in order to speed the time, we use the PCA to reduce the dimension, which will also lose some features of the images after flatten.

CNN

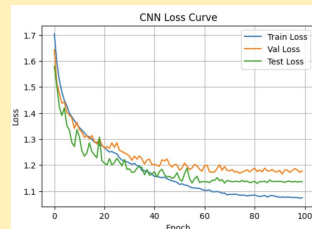
Workflow: We preprocess FER images through data augmentation (random horizontal flipping, rotation, and normalization) to enhance robustness and reduce overfitting. Subsequently, the dataset is divided into a training set, a validation set and a test set. The convolutional neural network (CNN) was trained for 100 rounds using Stochastic gradient Descent (SGD) and label smoothing, and the best model was selected based on the performance of the validation set. Finally, the model is evaluated on the test set and a confusion matrix is generated to analyze the performance of each category.



This dataset contains noise interference and visually blurred images, making certain emotions difficult to distinguish. The model is relatively small in scale, which limits its ability to capture fine facial details.

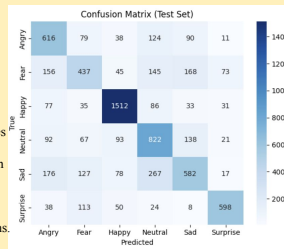


Neural network architecture: Our model adopts three convolutional modules, each of which contains two 3×3 convolutional layers (BN + ReLU) followed by 2×2 max-pooling. The number of channels increases successively: $32 \rightarrow 64 \rightarrow 128$, while the size of the feature map is reduced from 48×48 to 6×6 . The flattened output (4608 units) is processed through a 512-unit fully connected layer with dropout and eventually enters a 6-class classifier.



The model converges smoothly with no major overfitting. Training and validation loss decrease consistently across 100 epochs.

The confusion matrix shows that this model performs particularly well in identifying fear, neutrality and happiness emotions. Most mistakes are concentrated among emotions such as anger, sadness and surprise, whose visual features are more similar. Overall, convolutional neural networks can effectively capture the prominent features of distinct emotions, but they still struggle when dealing with subtle or ambiguous emotions.



ResNet18

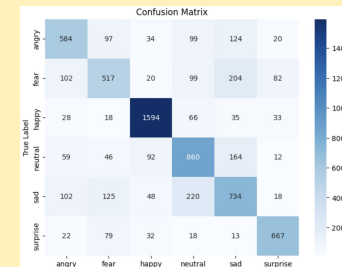
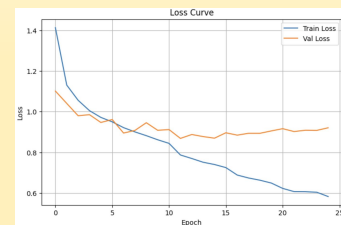
Model	Accuracy
ResNet18	71.37%
DenseNet	71.06%
DLA	70.93%
ResNet134	70.61%
DPN	70.34%
SENet18	70.32%
VGG19	69.47%

ResNet18 is a kind of CNN based on residual block. We used it because it has the best performance in single model. The pre-trained model we used here is based on ImageNet. We tried it with several optimizers and schedulers.

Model	Accuracy	Loss
Baseline CNN	0.41	1.2609
Final CNN	0.64	1.1361
Adam+None	0.68	0.9400
SGD+RLRP	0.70	0.9200

From the table, the workflow is that we added a ResNet(Adam only). We then added a scheduler and changed the optimizer.

Since the ResNet with SGD+RLRP has the best accuracy, we used it in the following parts. In this graph, it is clear that as the epoch increases, there's a trend to overfit. So we set it to be low.



From the confusion matrix, the model did much better than KNN and slightly better than CNN in the left. However, the limitation of this part is also obvious, there are more strategies for adjusting the learning rate such as cosine annealing and the one used here might not be the best one. Also, we did not combine these models together. Moreover, we did not use N-crop to provide a stable performance. In this question, we did not spend a lot of time adjusting parameters compared to the last one.